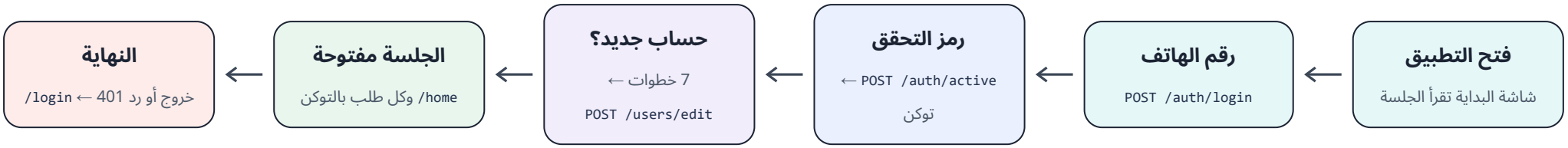
ميزة المصادقة auth من الألف إلى الياء

كيف يدخل الطالب برقم هاتفه ورمز تحقق، وكيف يُكمل حسابه إن كان جديداً، وأين تُحفظ الجلسة على الجهاز، ومتى تنتهي. كل صفحة مخطط واحد، والأسماء فيه هي أسماء الملفات والدوال في الكود نفسه.



بلا كلمة مرور
الدخول والتسجيل بالهاتف ورمز من 4 أرقام فقط.

3 شاشات
`/login` و`/otp` و`/register`، بعد شاشة البداية `/`.

11 + 2
إحدى عشرة حالة استخدام يقودها Cubit اثنان.

10 مسارات API
سبعة للحساب وثلاثة لقوائم التسجيل.

58 اختباراً
في 8 ملفات تحت `test/features/auth`، وكلها تنجح.

محتوى الملف

02 **رحلة المستخدم** — من فتح التطبيق حتى الرئيسية، وكل تفرّع

03 **خريطة الطبقات** — كل ملف في مكانه من Clean Architecture

04 **طلب رمز التحقق** — تسلسل `requestOtp` من الزر إلى الخادم

05 **التحقق من الرمز** — تسلسل `verifyOtp` وحفظ التوكن والتفرّع

06 **إكمال الحساب** — خطوات `RegisterPage` الثماني وقوائم الخادم

07 **إرسال التسجيل** — تسلسل `register` وتحويل الحقول إلى `users/edit`

08 **حالات الـ Cubit** — انتقالات `AuthState` ومن يستمع لكل حالة

09 **الجلسة والشبكة والأخطاء** — `SharedPreferences` و`ApiClient` و`guard`

10 **حارس المسارات والخروج** — `redirect` وانتهاء الجلسة بـ 401 وتسجيل الخروج

11 **ما بعد الدخول** — الملف الشخصي، ومن يستخدم حالات الاستخدام

12 **المرجع** — مسارات الـ API، النموذج، الحقن، الاختبارات

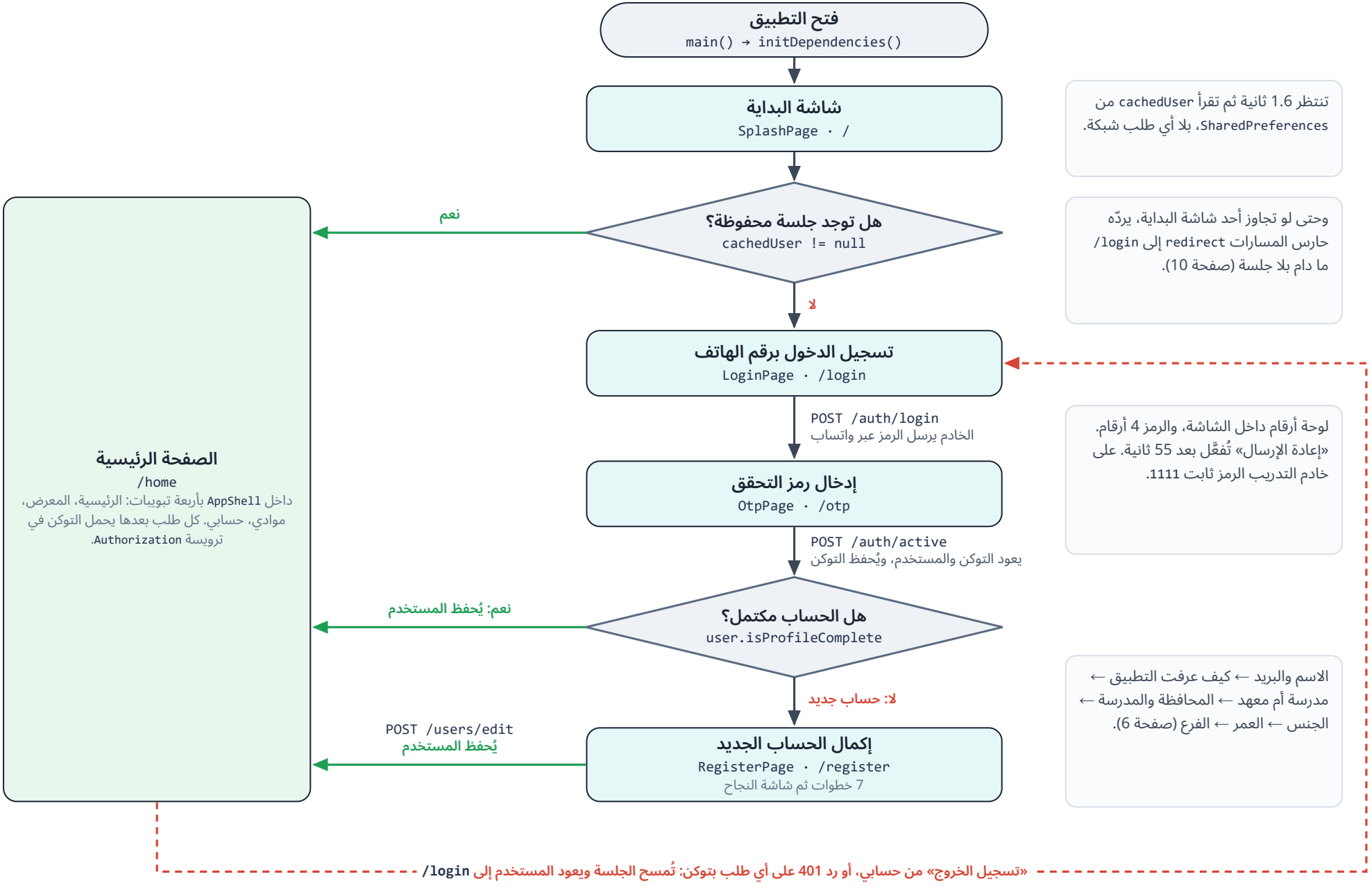
دليل الألوان

- العرض Presentation
- المجال Domain
- البيانات Data
- المشترك `core`
- الخادم والتخزين
- نجاح
- فشل أو خروج
- استدعاء
- رد أو نتيجة

المصدر: الكود الحالي في `lib/features/auth` و`lib/core`

02 رحلة المستخدم من فتح التطبيق إلى الرئيسية

الصورة الكبيرة قبل التفاصيل: ثلاث شاشات، وسؤالان يحددان الطريق، وطريق واحد للعودة.



03 خريطة الطبقات والملفات

34 ملفاً في `lib/features/auth` موزعة على ثلاث طبقات، وتستعين بأدوات `lib/core`.

قاعدة الاعتماد: العرض والبيانات يعتمدان على المجال، والمجال لا يعتمد على أحد.

ترتيب البناء (`FEATURE_WORKFLOW.md`): المجال ← البيانات ← الـ Cubit ← الحقن والمسار والواجهة.

العرض Presentation `presentation/`

الصفحات

`RegisterPage` `OtpPage` `LoginPage`

الحالة

`AuthState` `AuthCubit`

`RegistrationOptionsCubit`

`RegistrationOptionsState`

عناصر الواجهة

`LookupSheet`

ومن `core/widgets/auth_widgets.dart`:

`AuthField` `AuthHeader`

`RegisterProgress` `NumericKeypad`

`TermsSheet`

لا تعرف الخادم ولا التخزين: ترسل ما فعله المستخدم إلى الـ Cubit وتستمع لحالته.

← الـ Cubit يستدعي حالة الاستخدام

المجال Domain `domain/`

الكيانات

`RegistrationDetails` `School` `User`

`ProfileUpdate` `Gender` `AppSource`

`SchoolKind` `LookupOption` `ProfileImage`

العقود (واجهات مجردة)

`AuthRepository`

`RegistrationOptionsRepository`

حالات الاستخدام (11)

`Register` `VerifyOtp` `RequestOtp`

`RefreshProfile` `GetCurrentUser`

`UpdateProfileImage` `UpdateProfile`

`GetSchools` `GetGovernorates` `Logout`

`GetBranches`

بلا Flutter أو Dio أو تخزين. كل دالة تعيد `Either<Failure, T>`.

← حالة الاستخدام تستدعي العقد المجرد

→ التنفيذ يعتمد على العقد `implements`

البيانات Data `data/`

تنفيذ العقود

`AuthRepositoryImpl`

`RegistrationOptionsRepositoryImpl`

مصادر البيانات

`AuthRemoteDataSource`

`AuthLocalDataSource`

`RegistrationOptionsRemoteDataSource`

النماذج

`AuthSessionModel` `UserModel`

`LookupOptionModel`

تتحقق بـ `Validators` قبل الشبكة، وتلف كل استدعاء بـ `guard` فيصير الاستثناء `Failure`.

← المصادر تكلّم الخادم والجهاز

الخارج

الخادم

`Laravel API`

العنوان في `ApiConfig.baseUrl`

الجهاز

`SharedPreferences`

`auth.token`

`auth.user`

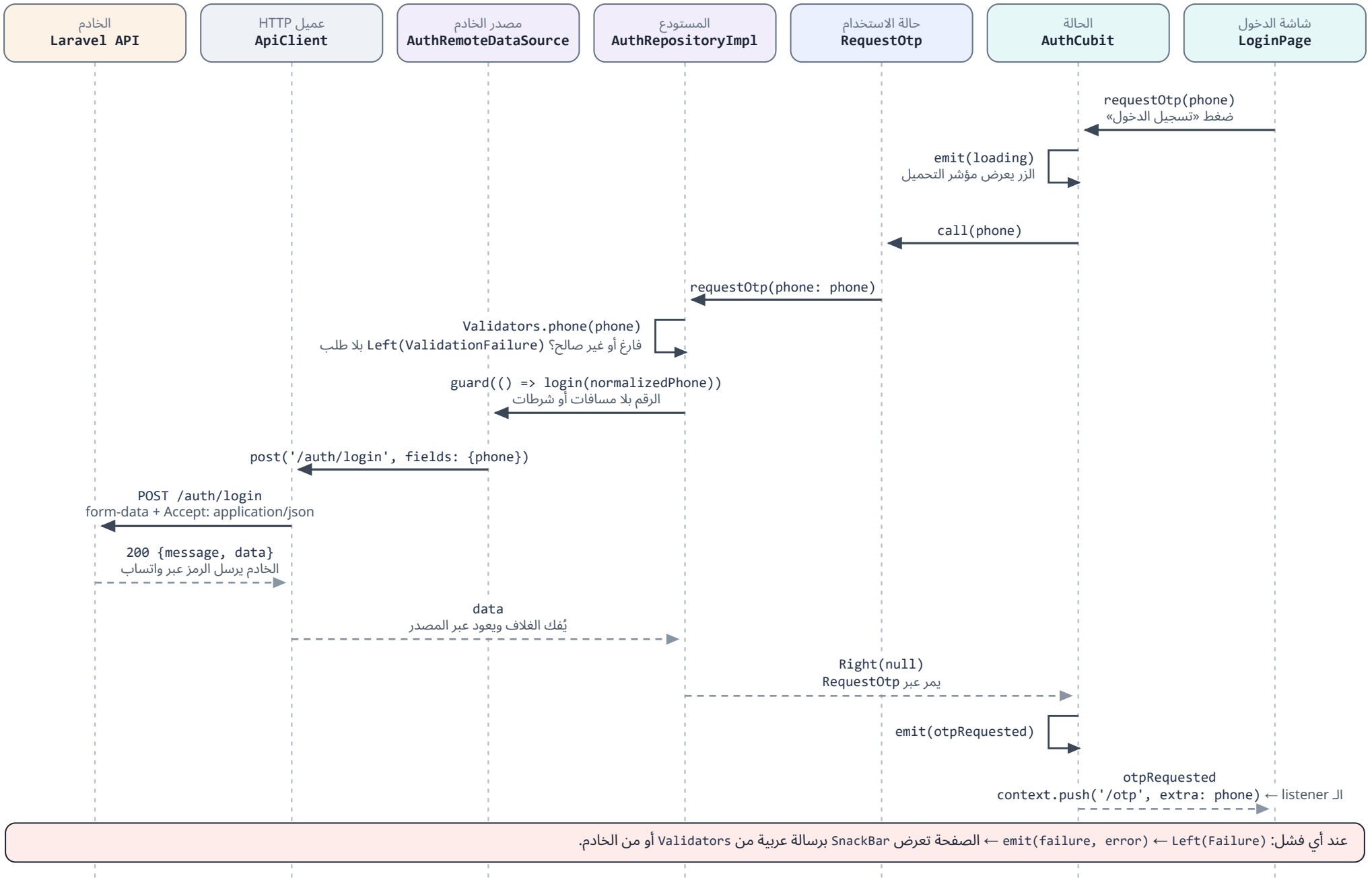
`auth.device_id`

المشترك `lib/core`

- **`ApiClient`** عميل HTTP الوحيد: التوكن، فك الغلاف، تحويل الأخطاء
- **`guard()`** يحوّل الاستثناء إلى **`Failure`**
- **`Validators`** قواعد الهاتف والرمز والبريد
- **`SessionExpiry`** انتهاء الجلسة عند 401
- **`UseCase`** · **`NoParams`** شكل موحد لحالات الاستخدام
- **`injection.dart`** حقن GetIt لكل ما سبق
- **`app_router.dart`** مسارات GoRouter وحارسها
- **`Failure`** أربعة أنواع: خادم، تخزين، تحقق، مصادقة

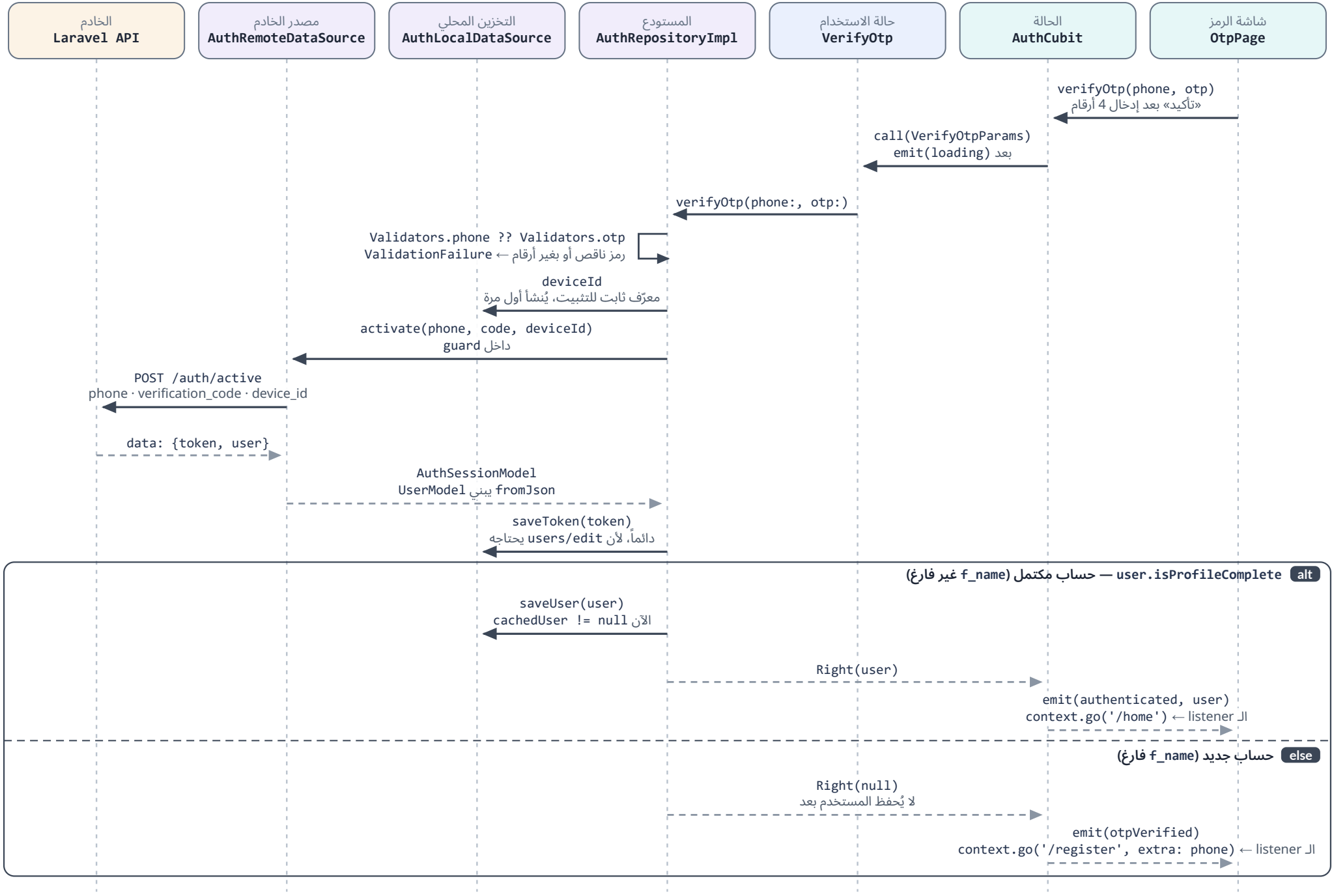
04 طلب رمز التحقق خطوة بخطوة

ما يحدث بين ضغط زر «تسجيل الدخول» وفتح شاشة الرمز، عبر كل طبقة.



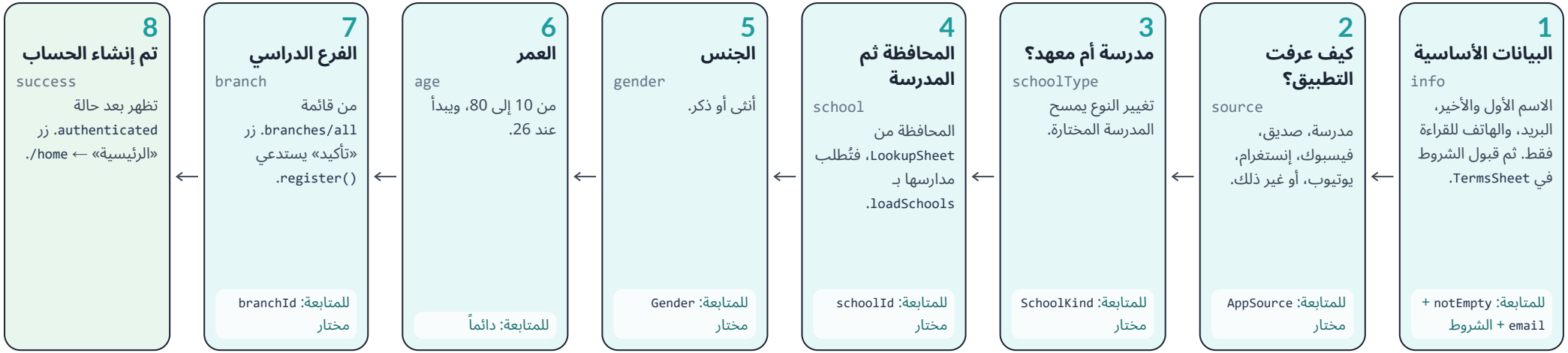
05 التحقق من الرمز وفتح الجلسة

هنا يتفرع الطريق: حساب قديم يدخل الرئيسية، وحساب جديد يذهب لإكمال بياناته.



06 إكمال الحساب: الخطوات والقوائم

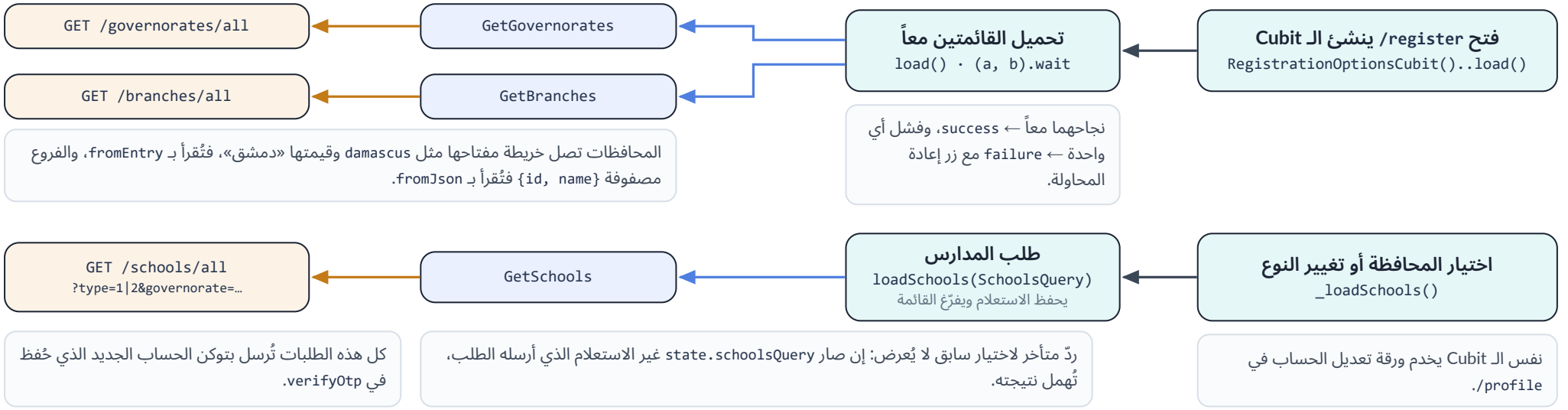
صفحة واحدة `RegisterPage` تعرض ثماني خطوات، وتجلب المحافظات والمدارس والفروع من الخادم.



الخطوات حالة محلية في `_RegisterPageState` (`enum _RegisterStep`)، لا في الـ Cubit: لا يُرسل شيء للخادم قبل الخطوة 7.

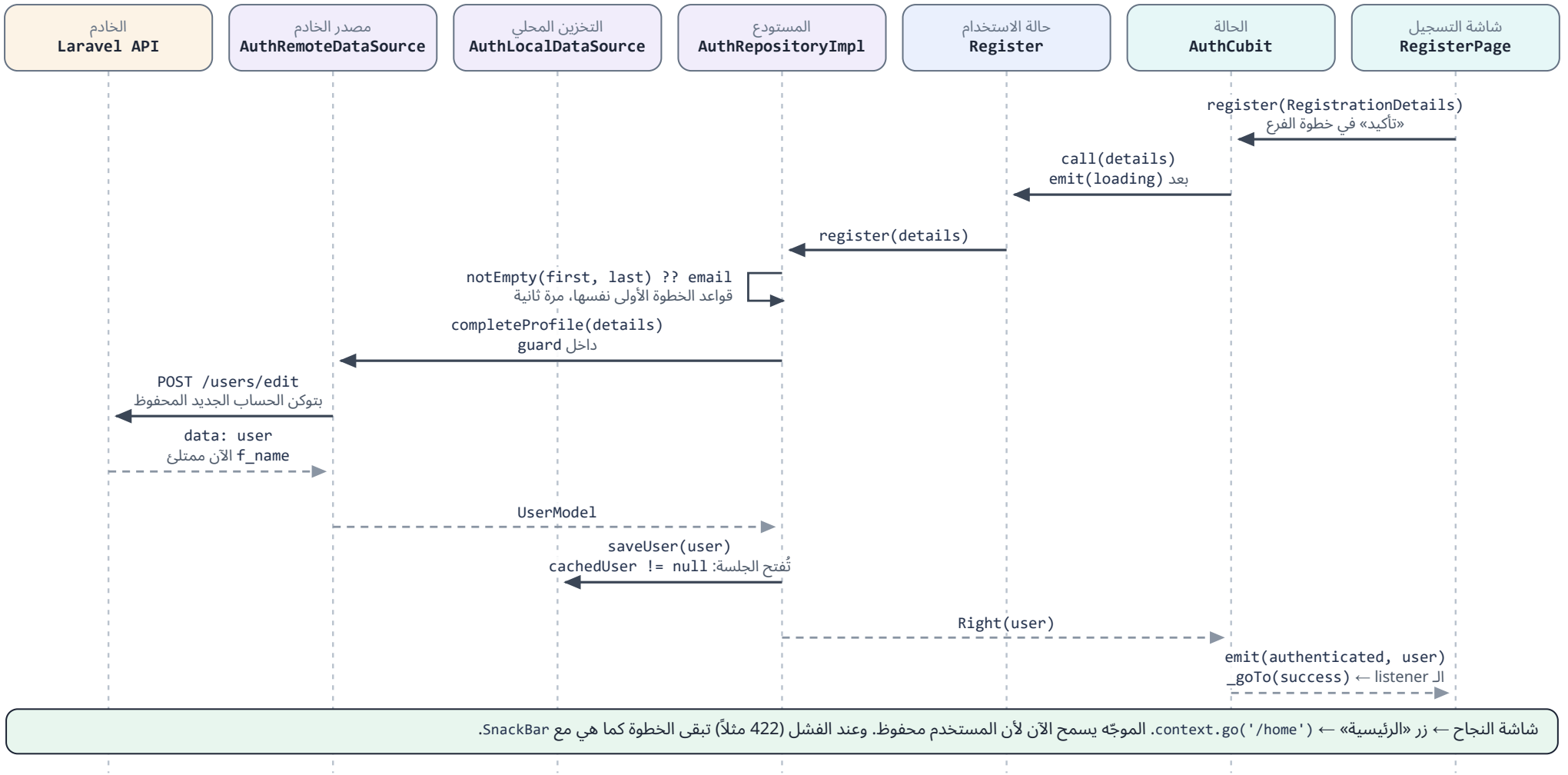
زر الرجوع: من الخطوة 1 ← `context.go('/login')`، ومن غيرها ← الخطوة السابقة بـ `_goBack()`.

قوائم الاختيار من الخادم — `RegistrationOptionsCubit`



07 إرسال التسجيل وتحويل الحقول

زر «تأكيد» في الخطوة السابعة هو أول طلب يغيّر الحساب على الخادم.

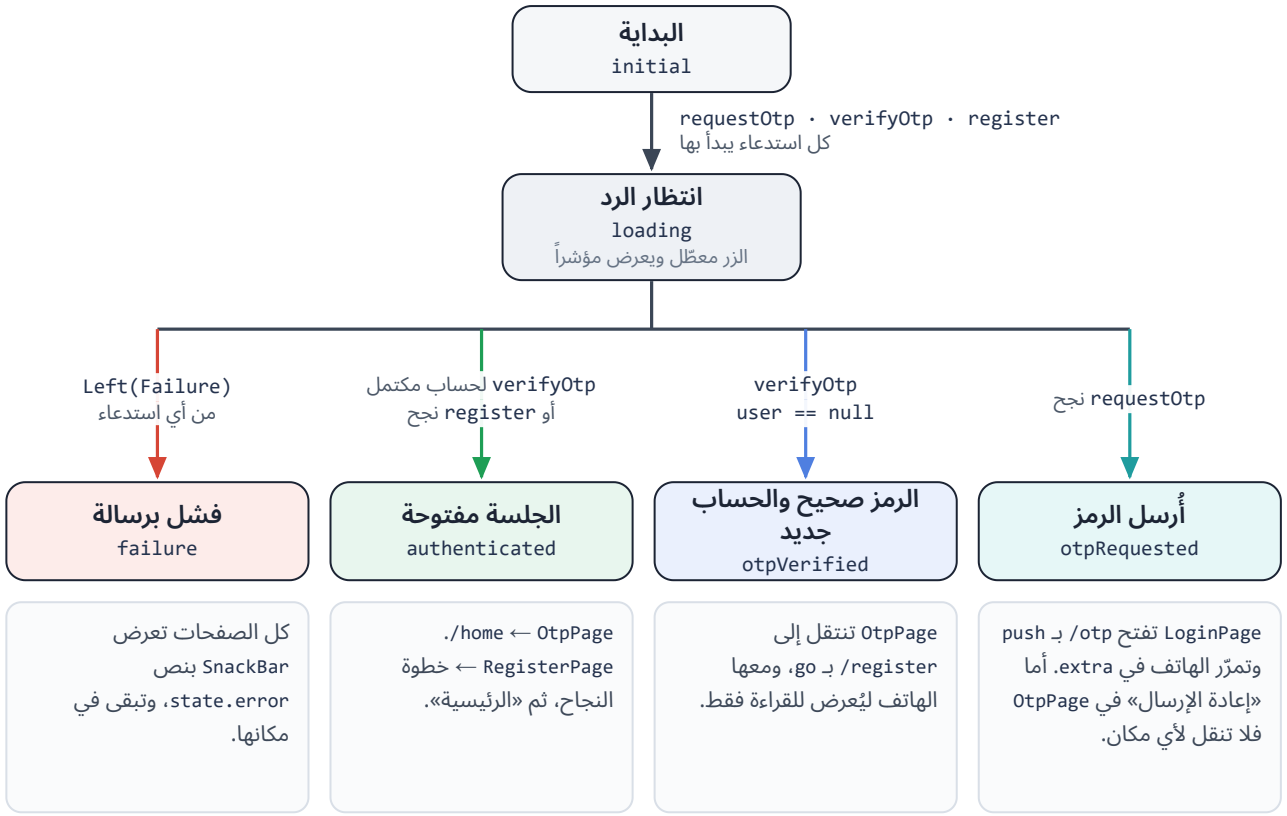


حقل users/edit	f_name	l_name	email	school_id	branch_id	gender	age	know_about_app
من RegistrationDetails	firstName	lastName	email	schoolId	branchId	gender	age	source
القيمة المرسلة	النص بلا مسافات طرفية	النص بلا مسافات طرفية	اختياري في الخادم	معرّف من schools/all	معرّف من branches/all	female ← 0، male ← 1	رقم من 10 إلى 80	فيسبوك 1، إنستغرام 2، صديق 3، مدرسة 4، يوتيوب وغير ذلك 5

الهاتف لا يُرسل لأن الخادم عرف صاحبه من التوكن. التحويل في AuthRemoteDataSourceImpl (_genders و_sources)، فلا تعرف طبقة المجال أرقام الخادم.

08 حالات الـ Cubit ومن يستمع إليها

`AuthStatus` ستّ قيم. الصفحة لا تقرر شيئاً بنفسها: تستمع للحالة في `BlocConsumer` وتنتقل.



Cubit جديد لكل شاشة

`AuthCubit` مسجل بـ `registerFactory`، فلكل من `/login` و`/otp` و`/register` نسخة خاصة. لذلك ينتقل الهاتف بين الشاشات عبر `extra`، لا عبر الحالة.

حارس `_emit`

الرد قد يصل بعد إغلاق الصفحة ومعها الـ Cubit، فيُهمل إن كان `isClosed` بدل أن يرمي استثناءً.

`error` لا يُورَّث

`copyWith` يمسح الرسالة في كل انتقال لا يصرّح بها، فلا تبقى رسالة قديمة بعد محاولة ناجحة.

`isLoading`

`status == loading`. يعطّل `AuthPrimaryButton` ويعرض مؤشر التحميل، فلا يُرسل الطلب مرتين.

حالة `RegistrationOptionsCubit` — مساران مستقلان في حالة واحدة

`status` — المحافظات والفروع معاً

`initial` ← `loading` ← `success` أو `failure`

`load()` بالتوازي؛ فشل أي قائمة يعني `failure` مع إعادة المحاولة.

`schoolsStatus` — المدارس وحدها

`initial` ← `loading` ← `success` أو `failure`

`loadSchools(query)` يحفظ `schoolsQuery`، والرد الذي لا يطابقه يُهمل.

09 الجلسة على الجهاز، والشبكة، والأخطاء

أين يعيش التوكن، وكيف يخرج كل طلب ويعود، وكيف يصير أي خطأ رسالة عربية واحدة.

الجلسة على الجهاز — `SharedPrefsAuthLocalDataSource`

المفتاح	ما فيه	يُكتب في	يُمسح
`auth.token`	توكن الخادم `id|token`	`verifyOtp`، دائماً	`clear()`
`auth.user`	`UserModel.toJson()` بمفاتيح الخادم	`verifyOtp` لحساب مكتمل، `register`، `refreshProfile`، `updateProfile`، `updateProfileImage`	`clear()`
`auth.device_id`	16 بايتاً عشوائياً بصيغة hex	أول قراءة لـ `deviceId`	لا يُمسح أبداً

- القراءة متزامنة (`token` و`cachedUser`)، لأن `ApiClient` يقرأ التوكن مع كل طلب، والموجّه يقرأ المستخدم عند كل انتقال.
- نسخة تالفة من `auth.user` تُقرأ `null`، أي كأن لا جلسة، بدل أن يتعطل التطبيق.
- «مسجّل الدخول» يعني وجود `auth.user`، لا وجود التوكن وحده.

طريق كل طلب في `ApiClient`



المعترض يقرأ التوكن لحظة الإرسال بـ `readToken()`، فيأخذ كل طلب أحدث توكن محفوظ. المهلة 15 ثانية للاتصال و20 للرد.



ونجاح يحمل رداً بغير الشكل المتوقع (`TypeError` أو `FormatException`) يصير `ServerException`: «رد غير متوقع من الخادم».



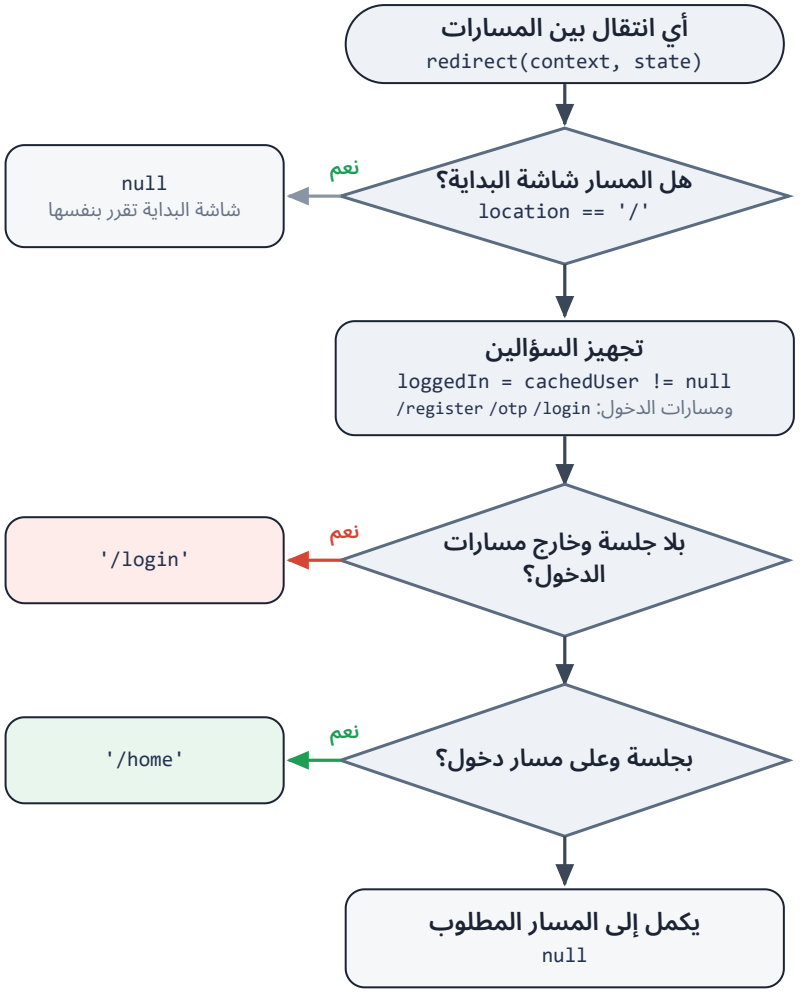
الـ Cubit لا يرى استثناءً أبداً: إما `Right` أو `Left(Failure)` برسالة عربية جاهزة للعرض.

ما حدث	يحوّله `ApiClient._mapError` إلى	الرسالة التي يراها المستخدم	يحوّله `guard` إلى
لا رد: انقطاع أو مهلة	`ServerException`	تعذّر الاتصال بالخادم، تحقق من اتصالك بالإنترنت	`ServerFailure`
401	`AuthException`، ويستدعي `onUnauthorized()` إن كان الطلب بتوكن	انتهت الجلسة، سجّل الدخول من جديد	`AuthFailure`
422	`ValidationException`	أول رسالة في `errors`، بلا «(and N more errors)»	`ValidationFailure`
404	`ServerException`	المحتوى المطلوب غير موجود	`ServerFailure`
غير ذلك، مثل 400 لرمز خاطئ	`ServerException`	`message` من الخادم كما هي (عربية)	`ServerFailure`
فحص `Validators` قبل الشبكة	لا طلب أصلاً	مثل «رقم الهاتف غير صحيح» و«رمز التحقق غير مكتمل»	`ValidationFailure` مباشرة
استثناء آخر (التخزين مثلاً)	—	حدث خطأ غير متوقع	`ServerFailure`

10 حارس المسارات، وانتهاء الجلسة، والخروج

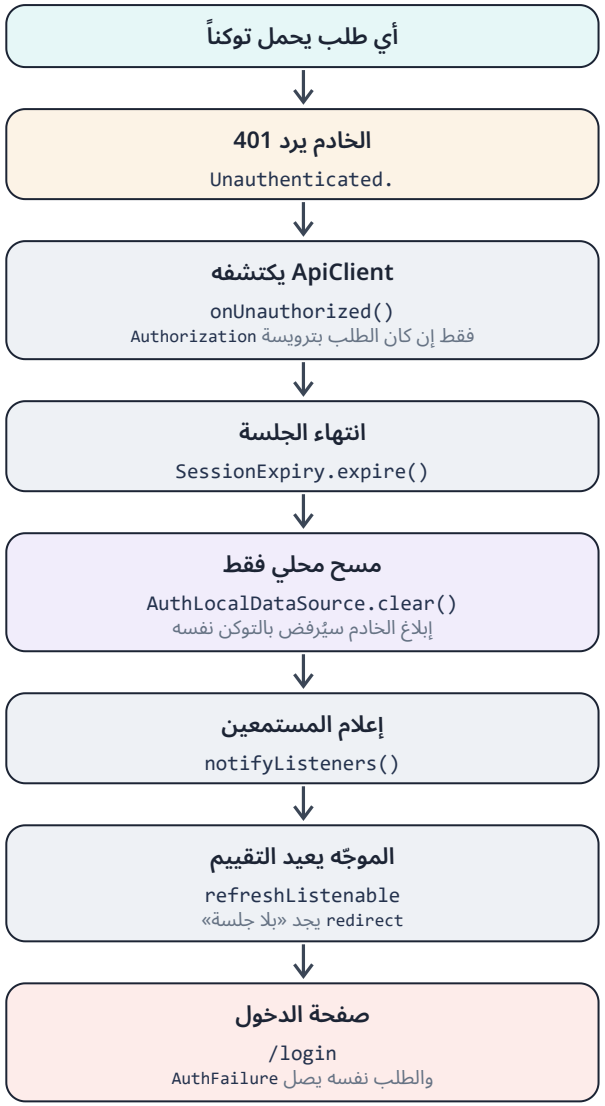
ثلاث طرق تحدد أين يقف المستخدم: سؤال عند كل انتقال، ورد 401 من الخادم، وزر الخروج.

حارس المسارات — `redirect` في `app_router.dart`

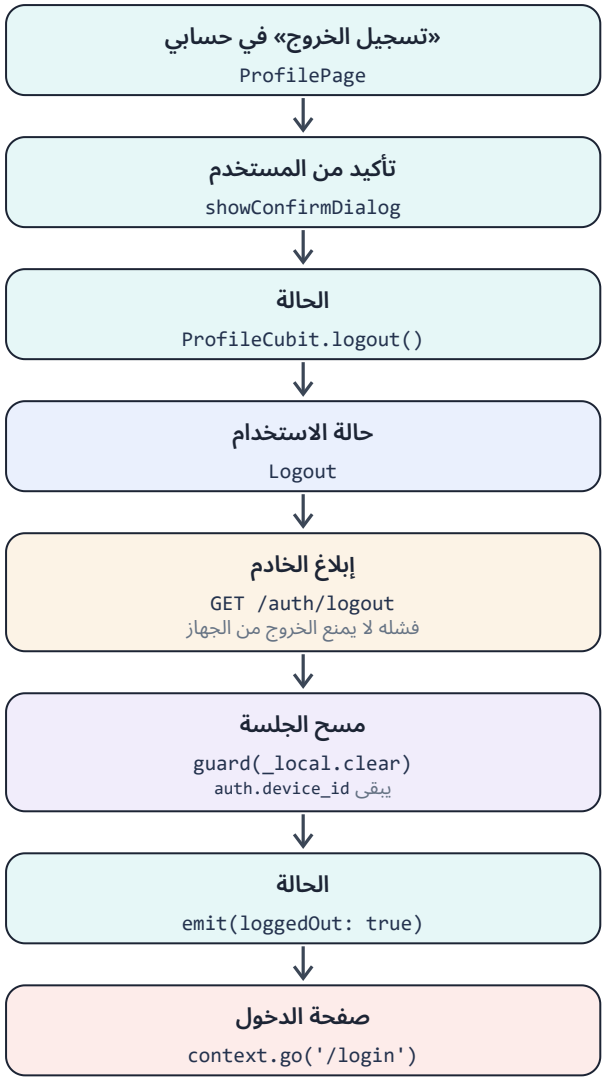


حساب جديد بعد `verifyOtp` له توكن، لكن لا مستخدم محفوظ له. فيُعدّ «بلا جلسة»، ويبقى `/register` متاحاً له، وأي مسار آخر يردّه إلى `/login`. بعد `register` يُحفظ المستخدم فيصير `/home` مسموحاً.

انتهاء الجلسة من الخادم — رد 401



تسجيل الخروج — من صفحة حسابي



11 ما بعد الدخول: الملف الشخصي ومن يستخدم الميزة

`AuthRepository` يبقى مصدر المستخدم في التطبيق كله، وكل تعديل يحدّث النسخة المحفوظة.

المستخدم الحالي

`GetCurrentUser`

`cachedUser` من الجهاز

↓

`Right(user)` أو `Right(null)` فوراً

بلا شبكة: تعرض الصفحات الأرقام المحفوظة أولاً.

تحديث من الخادم

`RefreshProfile`

يحفظ التوكن الحالي جانباً

↓

`GET /profile/get`

↓

`saveUser` ← إن لم يتغير التوكن

إن خرج المستخدم قبل وصول الرد، لا يُعاد حفظ جلسة أُغلقت.

تعديل البيانات

`UpdateProfile`

`notEmpty` للاسمين

↓

`POST /users/update`

↓

`saveUser` بما يعيده الخادم

الخادم يقبل هنا الاسم والمدرسة فقط، ويتجاهل البريد والفرع والعمر.

تغيير الصورة

`UpdateProfileImage`

النوع من الامتداد: jpg أو jpeg أو png

↓

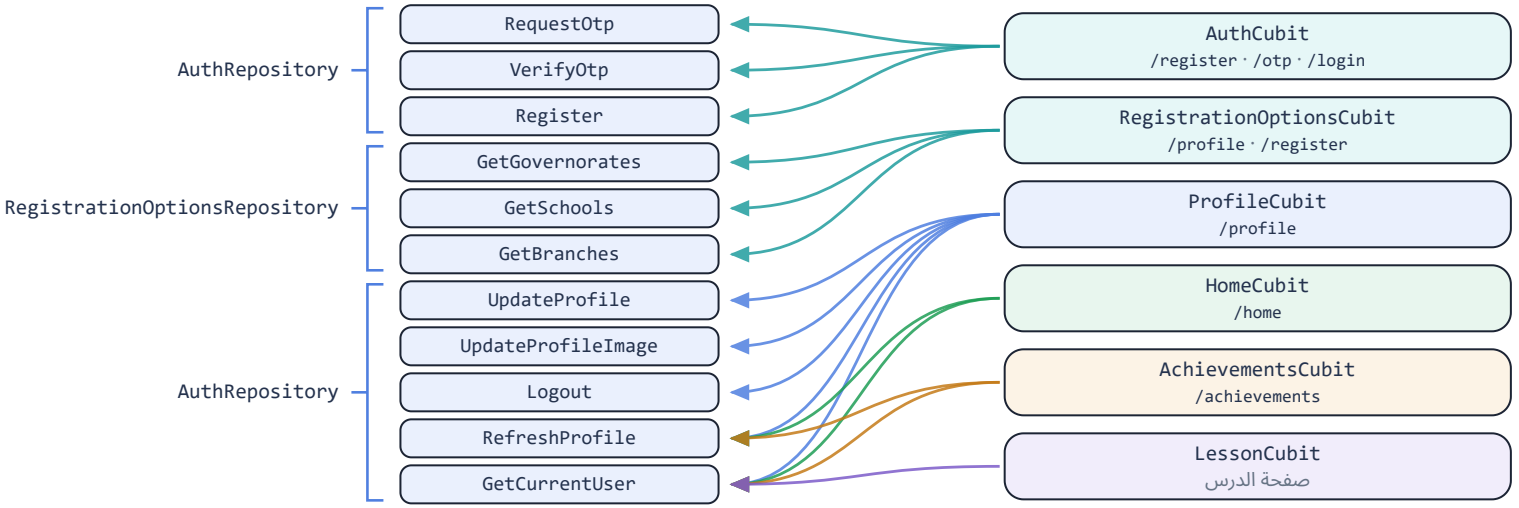
`POST /users/update-image`

↓

`saveUser` بالصورة الجديدة

صيغة أخرى ← «الصورة يجب أن تكون بصيغة JPG أو PNG» بلا طلب. الرفع `multipart` في الحقل `image`.

من يستخدم حالات الاستخدام الإحدى عشرة — ميزة auth تخدم ميزات أخرى بعد الدخول



خارج الـ Cubits: `SplashPage` والموجّه يقرآن `cachedUser` من `AuthRepository` مباشرة، و`ApiClient` و`SessionExpiry` يقرآن التوكن ويمسحان الجلسة من `AuthLocalDataSource`.

12 المرجع: الـ API، والنموذج، والحقن، والاختبارات

جداول للرجوع السريع، مأخوذة من الكود كما هو.

مسارات الـ API العشرة — كلها عبر `ApiClient`، والرد `{message, data}`

الطلب	يرسل	يعيد في `data`	يستدعيه
`POST /auth/login`	`phone`	—	`requestOtp`
`POST /auth/active`	`phone`، `verification_code`، `device_id`	`{token, user}`	`verifyOtp`
`POST /users/edit`	ثمانية حقول (صفحة 7)	المستخدم مكتملاً	`register`
`GET /profile/get`	—	المستخدم مع المدرسة والنقاط	`refreshProfile`
`POST /users/update`	`f_name`، `l_name`، `school_id`	المستخدم	`updateProfile`
`POST /users/update-image`	(multipart) `image`	المستخدم	`updateProfileImage`
`GET /auth/logout`	—	—	`logout`
`GET /governorates/all`	—	خريطة: المفتاح ← الاسم	`getGovernorates`
`GET /schools/all`	`type`، `governorate` (اختياريان)	`[{id, name}]`	`getSchools`
`GET /branches/all`	—	`[{id, name}]`	`getBranches`

الاختبارات — 58 اختباراً تنجح كلها: `flutter test test/features/auth`

الملف	العدد	ماذا يثبت
`auth_repository_impl_test.dart`	20	التحقق قبل الشبكة، الحفظ، الفشل، الخروج، سباق `refreshProfile`
`auth_remote_data_source_test.dart`	10	المسارات والحقول، وتحويل 401 و422 والانقطاع
`auth_cubit_test.dart`	7	الانتقالات، الرسالة لا تعلق، الإغلاق قبل الرد
`user_model_test.dart`	6	اكتمال الحساب، المدرسة، `toJson` ثم `fromJson`
`registration_options_remote_data_source_test.dart`	5	المحافظات من الخريطة، مرشّحات المدارس
`auth_local_data_source_test.dart`	4	الحفظ، المسح، ثبات معرّف الجهاز، النسخة التالفة
`registration_options_cubit_test.dart`	4	التحميل المتوازي، الرد المتأخر
`register_page_test.dart`	2	التسجيل بمعرّفات الخادم، محافظة بلا مدارس

من JSON إلى `User` — `UserModel.fromJson`، و`toJson` يكتب بالمفاتيح نفسها

مفتاح الخادم	حقل `User`	ملاحظة
`id`	`id`	يُحوَّل إلى نص
`phone`	`phone`	
`f_name · l_name`	`firstName · lastName`	و`name` هو الاثنان معاً
`f_name`	`isProfileComplete`	غير فارغ ← الحساب مكتمل
`email`	`email`	
`image`	`avatarUrl`	
`school`	`School(id, name, governorateId)`	غائب في رد `auth/active`
`points · medals`	`points · medalsCount`	0 إن غاب
`subject_count`	`enrolledCoursesCount`	
`semester_count`	`semestersCount`	

الحقن — `lib/core/di/injection.dart`

